

Sprint 3 — Cybersecurity

DevSecOps aplicado à solução Ford VIN Share

Desafio 02 — Impulsionando o VIN Share na América do Sul com Soluções Inteligentes

Item	Informação
Grupo	Ford VIN Share (Desafio 02)
Integrantes (nome — RM)	José Antonio Kretzer Rodriguez — RM555523 (3ESPW) Guilherme Machado Moreira — RM557290 (3ESPV) Enzo Almeida Santos Ramos — RM556900 (3ESPV) Gabriel de Mello Silva Fernandes — RM554421 (3ESPV)
Turma	3ESPV / 3ESPW
Repositório GitHub	https://github.com/JoseKretzer/ford-fiap-cyber
Entrega	27/09/2026

Documento único, organizado pelas quatro subetapas da Sprint: (1) Pipeline DevSecOps Integrado, (2) Segurança em Código e Infraestrutura, (3) Observabilidade, Monitoramento e Resposta e (4) Compliance, Riscos e Segurança Contínua. Cobre API, mobile, IoT, dados, ML e arquitetura.

Sumário

0. Contexto da solução, arquitetura e rastreabilidade dos requisitos

1. Pipeline DevSecOps Integrado (peso 3,0)

2. Segurança em Código e Infraestrutura (peso 2,5)

3. Observabilidade, Monitoramento e Resposta (peso 2,0)

4. Compliance, Riscos e Segurança Contínua (peso 2,5)

Anexo A. Estrutura do repositório e como executar

Anexo B. Lista completa dos testes automatizados

0. Contexto da solução e arquitetura de segurança

O **VIN Share** mede a porcentagem de veículos Ford que fazem manutenção na rede oficial. A solução do grupo ajuda as concessionárias a reter clientes no pós-venda com quatro partes: (a) um **app mobile** (Expo/React Native) em que consultores e gestores trabalham leads de serviço; (b) uma **API REST** (FastAPI) que expõe leads, indicadores de Service Share e previsões; (c) **telemetria de veículos conectados** recebida por MQTT e validada por um serviço de ingestão; e (d) um **modelo de Machine Learning** que estima a probabilidade de o veículo sair da rede Ford (churn).

A solução trata dados pessoais (nome, CPF, telefone), dados do veículo ligados ao proprietário (VIN, quilometragem, códigos de falha) e dados de desempenho das concessionárias. Cada controle descrito aqui existe no repositório e, sempre que possível, foi **executado e verificado** (testes, SAST, SCA, secret scanning, scan de IaC e simulação de ataques).

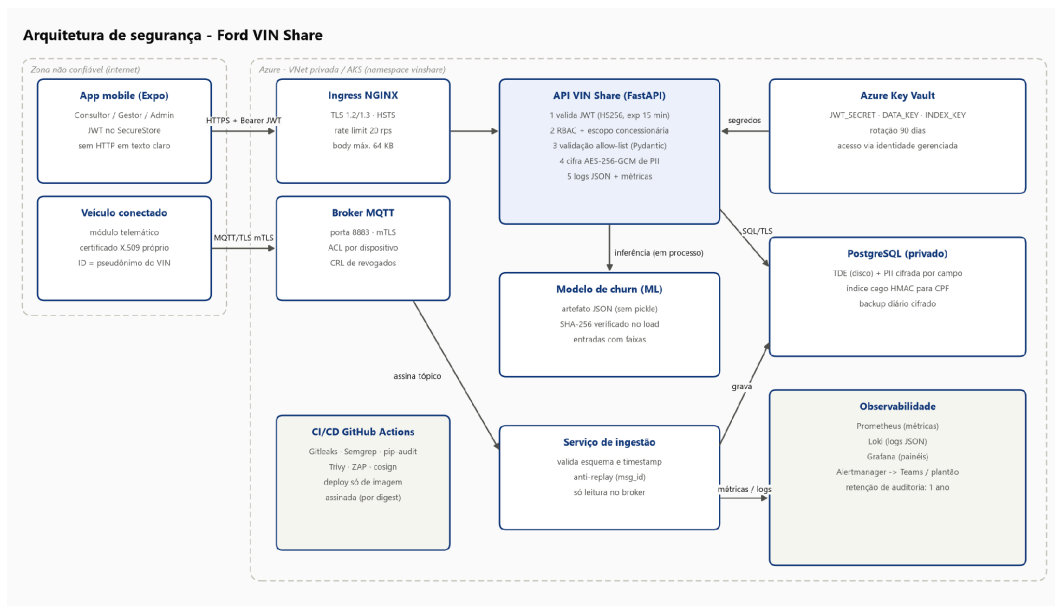


Figura 1 — Arquitetura de segurança: zonas de confiança, controles por componente e fluxos protegidos.

0.1 Perfis de acesso

O enunciado cita os perfis Brigadista, Gestor e Administrador. No contexto do VIN Share adaptamos os mesmos três níveis (operacional, gestão e administração) para a realidade das concessionárias:

Perfil	Equivalente no enunciado	Quem é	Escopo de dados
consultor	Brigadista (operacional)	Consultor de serviço da concessionária	Somente leads da própria concessionária
gestor	Gestor	Gestor de pós-venda da concessionária	Leads + Service Share + previsões da própria concessionária
admin	Administrador	Equipe Ford (analytics / segurança)	Todas as concessionárias, trilha de auditoria e usuários

0.2 Stack e escopo desta entrega

Camada	Tecnologia	O que foi entregue e verificado nesta Sprint
API	Python 3.13 · FastAPI · PyJWT · bcrypt · cryptography	Hardening completo + 49 testes de segurança executados
Mobile	Expo / React Native · expo-secure-store	Sessão segura, versão mínima obrigatória (426) e hardening do Android
IoT	MQTT (Mosquitto 2.0) · paho-mqtt	Broker TLS/mTLS/ACL/CRL, ingestão com anti-replay + 28 testes (inclui handshake mTLS real)

Camada	Tecnologia	O que foi entregue e verificado nesta Sprint
Dados	PostgreSQL (Azure) · Key Vault	PII cifrada por campo (AES-256-GCM), índice cego, rotina de backup
ML	Modelo de churn (regressão logística)	Artefato JSON com SHA-256 obrigatório em produção, entradas validadas, métricas de drift
Infra	Docker · Kubernetes (AKS)	Dockerfiles e manifest K8s com 0 misconfigurações no Trivy
CI/CD	GitHub Actions	Pipeline de 8 etapas, Actions fixadas por SHA, Dependabot
Observabilidade	Prometheus · Alertmanager · Loki · Alloy · Grafana	Métricas, logs JSON, 11 alertas validados contra ataques simulados, dashboard provisionado

0.3 Rastreabilidade: requisito do enunciado → onde está atendido

Subetapa	Requisito do enunciado	Onde (seção / arquivo)
1 Pipeline	Desenho do pipeline CI/CD com foco em segurança	1.1 (Figura 2) · .github/workflows/devsecops.yml
1 Pipeline	SAST (SonarQube, Semgrep)	1.2, 1.5, 1.7 · Semgrep + Bandit · .semgrep/
1 Pipeline	SCA (Dependabot, Snyk)	1.2, 1.5 · pip-audit, npm audit, Dependency Review, .github/dependabot.yml
1 Pipeline	Secret Scanning (GitGuardian, Gitleaks)	1.2, 1.5, 1.7 · Gitleaks · .gitleaks.toml
1 Pipeline	Container Security (Trivy)	1.2, 1.5 · Trivy imagem + IaC + SBOM
1 Pipeline	Explicação de como cada etapa reduz riscos	1.2 (com IDs de ameaça da seção 4.1) e 1.3
1 Pipeline	Como o pipeline seria executado no projeto Ford	1.4
2 Código	Criptografia local	2.1 · api/app/crypto.py · mobile/src/services/secureSession.ts
2 Código	Hardening de API (rate limit, validação de entrada, JWT seguro)	2.2 · api/app/
2 Código	Controle de acesso por perfil (Brigadista, Gestor, Administrador)	0.1, 2.3 · api/app/security.py
2 Código	Segurança MQTT/TLS para IoT	2.5 · iot/
2 Código	IaC Security (Dockerfile, Kubernetes YAML)	2.6 · api/Dockerfile, iot/Dockerfile, infra/k8s/api.yaml
2 Código	Trechos de código, prints, commits, explicações	Seção 2 inteira; 2.7 (correções reais); 1.5 (saídas das ferramentas)
3 Observab.	Logs estruturados (login, falhas, alterações críticas)	3.1 · api/app/logging_setup.py · exemplos_logs.jsonl
3 Observab.	Métricas e alertas (API, mobile, IoT, ML)	3.2 · observability/alert_rules.yml, loki-rules/
3 Observab.	Dashboards (Grafana ou equivalente)	3.3 (Figura 4) · observability/grafana/
3 Observab.	Plano de resposta: detecção → análise → contenção → erradicação → recuperação	3.4 (Figura 5 + 6 playbooks)
4 Compliance	Revisão final dos riscos (STRIDE + DevSecOps)	4.1 e 4.2
4 Compliance	OWASP ASVS · Mobile Top 10 · API Top 10	4.3, 4.4, 4.5
4 Compliance	LGPD (dados pessoais, telemetria, localização)	4.6
4 Compliance	Rotinas: dependências, testes, permissões, backup	4.7
4 Compliance	Documento consolidado + checklist de conformidade	Este documento + 4.8

1. Pipeline DevSecOps Integrado (peso 3,0)

Objetivo: mostrar como a segurança é incorporada ao pipeline, do commit ao deploy. O pipeline está em `.github/workflows/devsecops.yml` e roda no GitHub Actions a cada push e pull request. Nenhuma etapa de segurança é apenas informativa: se um *gate* falha, o merge é bloqueado pela proteção de branch e o deploy não acontece.

1.1 Desenho do pipeline

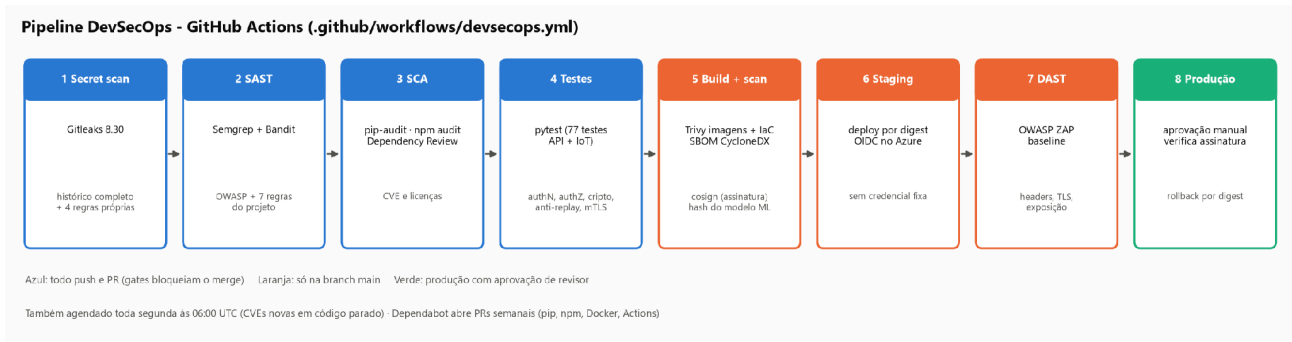


Figura 2 — Pipeline DevSecOps: 8 etapas, com gates bloqueantes do commit até a produção.

1.2 Etapas e riscos que cada uma reduz

Os IDs entre parênteses remetem às ameaças da revisão STRIDE (seção 4.1) e aos riscos do próprio ciclo DevSecOps (seção 4.2).

Etapas	Ferramenta	Quando roda	Bloqueia quando	Risco reduzido
1 Secret scanning	Gitleaks 8.30 (regras padrão + 4 do projeto)	Todo push/PR; todo o histórico	Qualquer segredo encontrado	Vazamento de JWT_SECRET, chaves AES/HMAC, senha MQTT, chave privada IoT (I-03, S-02)
2 SAST	Semgrep 1.178 (OWASP Top 10, Python, JWT, TS + 7 regras próprias) e Bandit 1.9	Todo push/PR	Achado ERROR/WARNING; Bandit médio+	JWT sem verificação (S-02), pickle no modelo (T-03), token no AsyncStorage (I-05), MQTT sem TLS (I-06), PII em log (I-04), rota sem auth (E-01)
3 SCA	pip-audit, npm audit, Dependency Review, Dependabot	Todo PR + semanal	CVE alta/crítica ou licença fora da lista	Dependência vulnerável ou maliciosa (T-04, OWASP M2)
4 Testes	pytest (77 testes: API + IoT)	Após 1-3	Qualquer teste falhando	Regressão em autenticação, autorização, validação, cripto, anti-replay e mTLS (S-01...E-01)
5 Build + container/IaC	Trivy 0.74 (2 imagens + config), SBOM CycloneDX, cosign, hash do modelo	Após testes	CVE alta/crítica corrigível ou misconfig alta/crítica	CVE do SO, container root, K8s inseguro (E-02); imagem trocada (DS-03); modelo trocado (T-03)
6 Deploy staging	azure/login (OIDC), kubectl	Só na main	Rollout não fica saudável	Credencial de nuvem fixa no GitHub (DS-02): não existe, o token OIDC dura minutos
7 DAST	OWASP ZAP baseline 0.15	Após staging	Regra FAIL (headers, HSTS, CSP)	Configuração insegura só visível em execução (I-04, D-01)
8 Produção	GitHub Environment + cosign verify	Após DAST + aprovação	Assinatura inválida ou sem aprovação	Deploy de imagem não verificada ou sem revisão (DS-03, DS-04)

1.3 Cobertura por componente (API, mobile, IoT, dados, ML, arquitetura)

Componente	Controles do pipeline que o cobrem
API (FastAPI)	Gitleaks, Semgrep (OWASP/JWT + regras próprias), Bandit, pip-audit, testes de segurança, Trivy imagem, ZAP

Componente	Controles do pipeline que o cobrem
Mobile (Expo)	Semgrep TypeScript + regra asyncstorage-token, npm audit, Dependabot npm; build do APK (EAS) consome só dependências aprovadas
IoT (broker + ingestão)	Semgrep regra mqtt-without-tls, Bandit, pip-audit (iot/requirements.txt), testes de anti-replay e de handshake mTLS, Trivy na imagem da ingestão
Dados	Gitleaks (chaves DATA_KEY/INDEX_KEY), testes de cifra em repouso e rotação de chave, Semgrep logging-sensitive-field
ML	Semgrep pickle-load-model, hash SHA-256 do modelo calculado no build e exigido pela API em produção, testes de faixas de entrada
Arquitetura / infra	Trivy config (Dockerfiles, Kubernetes), .trivyignore com aceite de risco datado, imagens por digest e assinadas

1.4 Como o pipeline é executado no projeto Ford

- **Desenvolvedor abre um pull request** para develop/main. A proteção de branch exige os checks secret-scan, sast, sca e tests verdes e 1 revisão de código. PRs vindos de *fork* rodam com o evento pull_request, sem acesso a segredos.
- **Etapas 1-3 rodam em paralelo.** Os resultados em SARIF aparecem na aba *Security* → *Code scanning* do GitHub, com arquivo e linha. O Dependency Review comenta no PR se alguma dependência nova tiver CVE.
- **Etapa 4** instala as dependências fixadas e roda as duas suítes (API e IoT). Os relatórios JUnit ficam como artefato do workflow.
- **Merge na main** dispara a etapa 5: o Trivy analisa Dockerfiles e manifests, as imagens são construídas, analisadas, recebem SBOM, são publicadas no GHCR e **assinadas** com cosign (keyless). O SHA-256 do modelo de ML é calculado nesta etapa.
- **Etapas 6-7:** deploy em staging pela imagem @sha256 (imutável), com o hash do modelo injetado em MODEL_SHA256, e varredura DAST com o ZAP.
- **Etapa 8:** o Environment production pede aprovação de um revisor de segurança; antes do deploy o pipeline **verifica a assinatura** da imagem. Se ela não foi gerada por este workflow na main, o deploy é abortado.
- **Agendamento semanal** (segunda, 06:00 UTC) roda tudo de novo, porque CVEs novas aparecem em código parado.

1.5 Execução real das ferramentas (evidência)

Antes de publicar o repositório, rodamos localmente as mesmas ferramentas e versões do pipeline. Os binários do Gitleaks e do Trivy foram baixados dos releases oficiais e **conferidos pelo SHA-256** do arquivo de checksums publicado. As saídas completas estão em docs/evidencias/.

Ferramenta	Escopo	Resultado	Evidência
Gitleaks 8.30.1	Repositório inteiro (~861 KB)	0 vazamentos. Controle positivo: 6/6 segredos falsos plantados detectados, incluindo as 4 regras próprias	gitleaks.json, gitleaks-controle.txt
Semgrep 1.178.0	53 arquivos (API, IoT, mobile, observabilidade)	0 achados com OWASP Top 10, Python, JWT, TypeScript + regras próprias	semgrep.json
Regras próprias Semgrep	Casos anotados (9 positivos, 6 negativos)	9/9 detectados, 0 falsos positivos	semgrep-regras.txt
Bandit 1.9.4	api/app + iot (1008 linhas)	0 achados (após corrigir o B101). 14 achados baixos no simulador de tráfego, triados como falso positivo	bandit.txt, bandit-completo.txt
pip-audit 2.10.1	api + iot requirements	0 vulnerabilidades conhecidas	pip-audit.txt
Trivy 0.74.0 (config)	2 Dockerfiles + manifest K8s	0 misconfigurações após corrigir o KSV-0013; KSV-0125 com aceite de risco datado	trivy-config*.txt
Trivy 0.74.0 (fs)	Dependências Python	0 CVEs (segunda opinião do SCA)	trivy-fs.txt
pytest	API + IoT	77/77 aprovados	pytest-.xml / .txt

Triagem do Bandit: os 14 achados de severidade baixa estão em observability/simular_trafego.py, uma ferramenta de teste que não vai para produção. São B311 (random usado para gerar tráfego, não para criptografia) e B105 (senha errada proposital para simular erro de digitação). Por isso o escopo do Bandit no pipeline é api/app e iot.

1.6 Trechos do workflow

Etapa 1 — Secret scanning em todo o histórico (Action fixada por SHA)

```
secret-scan:
  name: 1 · Secret scanning (Gitleaks)
  runs-on: ubuntu-latest
  permissions:
    contents: read
    security-events: write
  steps:
    - uses: actions/checkout@3d3c42e5aac5ba805825da76410c181273ba90b1 # v7.0.1
      with:
        fetch-depth: 0 # varre TODO o histórico, não só o último commit
        persist-credentials: false
    - name: Gitleaks
      run: |
        docker run --rm -v "$PWD:/repo" ghcr.io/gitleaks/gitleaks:v8.30.1 \
          git /repo --config /repo/.gitleaks.toml --redact \
          --report-format sarif --report-path /repo/gitleaks.sarif --exit-code 1
    - name: Publicar resultado na aba Security
      if: always()
      uses: github/codeql-action/upload-sarif@2892aa5e19bbd11bc0cff5427e3b750a04d9e3c2 # v4.38.2
      with:
        sarif_file: gitleaks.sarif
        category: gitleaks
```

Etapa 2 — As regras próprias são testadas antes de serem usadas

```
- name: Validar as regras próprias contra os casos de teste
  run: python .semgrep/verificar_regras.py
- name: Semgrep (regras OWASP + regras próprias do projeto)
  run: |
    semgrep scan --metrics=off \
      --config p/python --config p/owasp-top-ten --config p/jwt \
      --config p/typescript --config .semgrep/vinshare-rules.yml \
      --exclude .semgrep \
      --severity ERROR --severity WARNING \
      --text --sarif-output=semgrep.sarif --error # achados no log + SARIF para a aba Security
```

Etapa 5 — Hash do modelo de ML, exigido pela API em produção

```
- name: Hash do modelo de ML (vira MODEL_SHA256 no deploy; a API recusa modelo diferente)
  id: model
  run: echo "sha256=$(sha256sum api/app/churn_model.json | cut -d' ' -f1)" >> "$GITHUB_OUTPUT"
```

Etapa 8 — Produção só aceita imagem assinada por este workflow na main

```
- name: Verificar assinatura antes do deploy
  run: |
    IMAGE_API="ghcr.io/${GITHUB_REPOSITORY,,}/vinshare-api"
    cosign verify "${IMAGE_API}@${needs.build-and-scan.outputs.api_digest}" \
      --certificate-identity-regexp "^https://github.com/${github.repository}" \
      --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

1.7 Regras próprias (ameaças específicas do VIN Share)

Os rulesets públicos não conhecem os riscos do projeto. Por isso escrevemos 7 regras Semgrep e 4 regras Gitleaks ligadas às ameaças da seção 4.1. Cada regra Semgrep tem casos de teste anotados (.semgrep/vinshare-rules.py e .ts), validados no pipeline por verificar_regras.py.

Regras Semgrep do projeto (.semgrep/vinshare-rules.yml)

```
- id: pickle-load-model
  languages: [python]
  severity: ERROR
  message: "Não carregue modelos de ML com pickle/joblib (execução de código arbitrário, T-03). Use JSON/ONNX + hash."
  pattern-either:
    - pattern: pickle.load(...)
    - pattern: pickle.loads(...)
    - pattern: joblib.load(...)

- id: asyncstorage-token
  languages: [typescript, javascript]
  severity: ERROR
  message: "Token salvo no AsyncStorage (texto claro no aparelho). Use expo-secure-store (OWASP M9, I-05)."
  pattern: AsyncStorage.setItem($K, $TOKEN)
```

Regra Gitleaks para as chaves de criptografia de dados pessoais (.gitleaks.toml)

```
id = "vinshare-data-key"
description = "Chave AES/HMAC de dados pessoais gravada no código"
```

```
regex = '''(?i)(DATA_KEY_B64|INDEX_KEY_B64)\\s*[=:]\\s*["']?[A-Za-z0-9+/{40,}={0,2}'''
tags = ["crypto", "lgpd"]
```

1.8 Proteção do próprio pipeline (supply chain)

- **Actions fixadas pelo SHA do commit** (a tag fica só no comentário), porque tags podem ser reescritas (caso tj-actions/changed-files, março de 2025). O Dependabot github-actions abre PR com o SHA novo.
- **Menor privilégio:** permissions: contents: read por padrão, cada job pede só o que precisa, e persist-credentials: false no checkout.
- **Sem credenciais de nuvem guardadas:** login no Azure por OIDC (id-token: write).
- **Rastreabilidade:** SBOM CycloneDX, *provenance* mode=max e assinatura cosign de cada imagem.
- **Exceções com prazo:** o .trivyignore só aceita exceção com justificativa, dono e data de expiração. Quando a data passa, o pipeline volta a falhar.

2. Segurança em Código e Infraestrutura (peso 2,5)

Objetivo: mostrar práticas de segurança aplicadas diretamente no código e na infraestrutura. Os trechos abaixo são copiados dos arquivos do repositório. Os controles foram verificados por **77 testes automatizados (77 aprovados)** e pelas ferramentas da seção 1.5. As correções feitas a partir desses achados estão na seção 2.7.

2.1 Criptografia local

Os dados pessoais (CPF e telefone) são **cifrados por campo antes de serem gravados**, com AES-256-GCM. Quem tiver acesso de leitura ao banco ou a um backup vazado vê apenas texto cifrado.

- **AES-256-GCM** (cifra autenticada): qualquer alteração no texto cifrado é detectada e a leitura falha fechada.
- **Nonce aleatório de 96 bits** por operação; **contexto como AAD**, então um valor cifrado não pode ser movido de um campo para outro.
- **ID da chave no dado** (k1: . . .): permite rotacionar a chave sem perder os dados antigos (testado).
- **Índice cego HMAC-SHA256** para buscar pelo CPF sem guardá-lo em claro, com **separação de domínio** (idx: para CPF, vin: para o pseudônimo do VIN) mesmo usando a mesma chave.
- **Chaves fora do código** (Azure Key Vault → variável de ambiente); a API não sobe com chave ausente ou de tamanho errado.
- **No app mobile:** o token fica no expo-secure-store (Android Keystore / iOS Keychain), com `WHEN_UNLOCKED_THIS_DEVICE_ONLY` e `allowBackup: false`.

api/app/crypto.py — cifra e decifra de campos pessoais

```
def encrypt(self, plaintext: str, context: str) -> str:
    """Cifra um campo. `context` (ex.: 'customer.cpf') entra como AAD:
    um valor cifrado de um campo não pode ser copiado para outro."""
    nonce = os.urandom(12)
    ct = self._keys[self._active].encrypt(nonce, plaintext.encode(), context.encode())
    return f"{self._active}:{base64.b64encode(nonce + ct).decode()}"

def decrypt(self, token: str, context: str) -> str:
    try:
        kid, payload = token.split(":", 1)
        raw = base64.b64decode(payload)
        return self._keys[kid].decrypt(raw[:12], raw[12:], context.encode()).decode()
    except (ValueError, KeyError, InvalidTag) as exc:
        # Falha fechada: nunca devolve dado parcial ou adulterado
        raise DecryptionError("Falha ao decifrar campo protegido") from exc
```

2.2 Hardening da API

Controle	Implementação	Arquivo
JWT seguro	HS256 fixo (rejeita alg: none), segredo de 256 bits ou mais, expiração de 15 min, claims obrigatórios (iss, aud, exp, nbf, jti), revogação no logout, sem PII no payload	app/security.py
Senhas	bcrypt com custo 12, mínimo de 12 caracteres, tempo de resposta constante para usuário inexistente	app/security.py
Rate limit	Login: 5/min por usuário+IP e 20/min por IP. API: 100/min por cliente. Responde 429 com Retry-After	app/ratelimit.py, routes.py
Validação de entrada	Allow-list Pydantic: tipos, faixas, enums, regex; extra=forbid bloqueia mass assignment; rejeita caracteres de controle (log injection)	app/schemas.py
Tamanho do corpo	Limite de 64 KB (413), repetido no Ingress	app/main.py
Versão do app	X-App-Version abaixo de MIN_APP_VERSION (ou forjada) recebe 426	app/main.py
Cabeçalhos	HSTS, CSP default-src 'none', nosniff, X-Frame-Options DENY, no-store, Permissions-Policy	app/main.py
Erros	RFC 9457 (problem+json) sem stack trace; o valor enviado não é devolvido; request_id para o suporte	app/errors.py
Configuração	Falha no startup com segredo fraco, CORS * ou (em produção) sem MODEL_SHA256	app/config.py
Exposição	Swagger desligado em produção; /metrics não é roteado pelo Ingress	app/main.py, k8s/api.yaml

Controle	Implementação	Arquivo
Modelo de ML	Artefato JSON (sem pickle) com SHA-256 verificado antes de carregar; entradas com faixas	app/model.py

api/app/security.py — validação do JWT (algoritmo, audience, issuer e claims obrigatórios)

```
def validate(self, token: str) -> Principal:
    try:
        claims = jwt.decode(
            token,
            self._secret,
            algorithms=[ALGORITHM],
            audience=self._audience,
            issuer=self._issuer,
            options={"require": REQUIRED_CLAIMS},
            leeway=5,
        )
        role = Role(claims["role"])
    except jwt.ExpiredSignatureError as exc:
        raise ApiError(401, "token_expired", "Token expirado") from exc
    except (jwt.InvalidTokenError, ValueError) as exc:
        raise ApiError(401, "invalid_token", "Token inválido") from exc

    if claims["jti"] in self._revoked:
        raise ApiError(401, "token_revoked", "Token revogado")

    return Principal(claims["sub"], role, claims["dealer_id"], claims["jti"], claims["exp"])
```

api/app/routes.py — login com rate limit duplo, mensagem genérica e auditoria

```
for limiter, key, scope in ((app.login_limiter, f"{ip}|{body.username}", "login_user"),
                           (app.login_ip_limiter, ip, "login_ip")):
    allowed, retry_after = limiter.check(key)
    if not allowed:
        audit("auth.login.blocked", request, username=body.username, scope=scope)
        app.metrics.rate_limited.labels(scope=scope).inc()
        raise ApiError(429, "too_many_requests", "Muitas tentativas de login. Tente novamente mais tarde.",
                       {"Retry-After": str(retry_after)})

user = app.store.users.get(body.username)
if not verify_password(body.password, user.password_hash if user else None) or not user.active:
    audit("auth.login.failure", request, username=body.username)
    app.metrics.login.labels(result="failure").inc()
    # Mensagem genérica: não informa se o usuário existe
    raise ApiError(401, "invalid_credentials", "Usuário ou senha inválidos")
```

api/app/schemas.py — validação por allow-list das entradas do modelo de ML

```
class ChurnPredictionRequest(Strict):
    model: FordModel
    vehicle_age_years: int = Field(ge=0, le=30)
    km_since_last_service: int = Field(ge=0, le=200_000)
    months_since_last_visit: int = Field(ge=0, le=120)
    services_last_24m: int = Field(ge=0, le=50)
    warranty_active: bool
    connected_vehicle: bool
```

2.3 Controle de acesso por perfil

A autorização é feita **sempre no servidor**, em duas camadas: (1) a **permissão do perfil** para a operação e (2) o **escopo da concessionária** do recurso, que protege contra BOLA/IDOR (OWASP API1). O perfil que o app recebe no login serve só para montar o menu.

Permissão	consultor	gestor	admin	Endpoints
leads:read	✓ (própria)	✓ (própria)	✓ (todas)	GET /dealers/{id}/leads, GET /leads/{id}
leads:update	✓ (própria)	✓ (própria)	✓ (todas)	PATCH /leads/{id}
service_share:read	—	✓ (própria)	✓ (todas)	GET /dealers/{id}/service-share
predictions:create	—	✓	✓	POST /predictions/churn
audit:read	—	—	✓	GET /admin/audit-events
users:manage	—	—	✓	(gestão de usuários)

api/app/security.py — matriz de permissões (menor privilégio)

```
PERMISSIONS: dict[Role, set[str]] = {
    Role.CONSULTOR: {"leads:read", "leads:update"},
    Role.GESTOR: {"leads:read", "leads:update", "service_share:read", "predictions:create"},
    Role.ADMIN: {"leads:read", "leads:update", "service_share:read", "predictions:create",
                 "audit:read", "users:manage", "dealers:all"},
}
```

api/app/routes.py — proteção contra BOLA: lead de outra concessionária responde 404, sem revelar que existe

```
def _get_lead_scoped(request: Request, principal: Principal, lead_id: str) -> Lead:
    lead = request.app.state.store.leads.get(lead_id)
    if lead is None:
        raise ApiError(404, "not_found", "Lead não encontrado")
    if not principal.can_access_dealer(lead.dealer_id):
        # Responde 404 (e não 403) para não revelar que o lead existe em outra concessionária
        audit("authz.denied", request, user_id=principal.user_id, role=principal.role.value,
             permission="dealer_scope", target_dealer=lead.dealer_id)
        request.app.state.metrics.authz_denied.labels(permission="dealer_scope").inc()
        raise ApiError(404, "not_found", "Lead não encontrado")
    return lead
```

2.4 Mobile: sessão segura e versão mínima

O app guarda o token só no armazenamento seguro do sistema, usa apenas HTTPS e envia a própria versão. Quando uma versão tem falha de segurança conhecida, basta subir MIN_APP_VERSION na API: os aparelhos desatualizados recebem 426 e o app pede a atualização (OWASP Mobile M8).

mobile/src/services/secureSession.ts — 401 com token = sessão expirada; 426 = atualização obrigatória

```
// 401 só significa "sessão expirada" se um token foi enviado; no login é senha errada
if (response.status === 401 && token) {
    await clearSession();
    throw new SessionExpiredError("Sessão expirada. Faça login novamente.");
}
if (response.status === 426) {
    throw new UpdateRequiredError("Atualize o aplicativo para continuar.");
}
```

api/app/main.py — a API recusa versões antigas ou forçadas do app

```
elif app_version is not None and outdated:
    # App com falha de segurança conhecida: força atualização (OWASP Mobile M8)
    response = problem(request, 426, "upgrade_required",
                       "Versão do aplicativo não suportada. Atualize o app para continuar.")
```

2.5 Segurança MQTT/TLS para IoT

- **Somente TLS (porta 8883)**, versão mínima 1.2 e cifras ECDHE+AES-GCM. A porta 1883 não existe.
- **mTLS**: cada módulo telemático tem certificado X.509 próprio; o CN é o **pseudônimo do VIN** e vira o usuário na ACL.
- **ACL por dispositivo**: o veículo só publica em `vehicles/<seu-id>/telemetry`; a ingestão só lê.
- **Revogação**: `gerar_certificados.ps1 -Revogar <id>` revoga o certificado e regenera a CRL (playbook PB-04).
- **Ingestão desconfia da telemetria** (OWASP API10): tópico, 4 KB, esquema estrito, janela de 5 min, **anti-replay** por `msg_id` e odômetro que não pode regredir. Cada rejeição vira métrica e log.
- **Minimização**: só odômetro, códigos de falha e vida do óleo. Não há GPS.

iot/mosquitto/mosquitto.conf — broker com TLS obrigatório, mTLS e CRL

```
per_listener_settings false
allow_anonymous false

listener 8883
protocol mqtt
tls_version tlsv1.2
cafile /mosquitto/certs/ca.crt
certfile /mosquitto/certs/broker.crt
keyfile /mosquitto/certs/broker.key
require_certificate true           # cada módulo telemático tem seu próprio certificado
use_identity_as_username true      # CN do certificado = ID do dispositivo (usado na ACL)
ciphers
ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256

# Certificados revogados (dispositivo roubado/comprometido) são recusados
crlfile /mosquitto/certs/crl.pem
```

```
acl_file /mosquitto/config/acl.conf
```

iot/mosquitto/acl.conf — isolamento entre veículos

```
pattern write vehicles/%u/telemetry
pattern read vehicles/%u/commands

# Serviço de ingestão (certificado CN=ingestion-service) lê a telemetria de todos, sem escrever.
user ingestion-service
topic read vehicles/+/telemetry
```

iot/ingestion.py — janela de tempo, anti-replay e coerência do odômetro

```
if data["ts"] < now - self.max_age:
    return self._reject("mensagem_antiga", device)
if data["ts"] > now + self.max_future:
    return self._reject("timestamp_futuro", device)

self._seen = {k: t for k, t in self._seen.items() if t > now - self.replay_ttl}
key = (device, data["msg_id"])
if key in self._seen:
    return self._reject("replay", device)

if data["odometer_km"] < self._odometer.get(device, 0):
    return self._reject("odometro_regrediu", device)
```

Handshake mTLS testado de verdade (iot/tests/test_mtls.py): o teste gera uma PKI, sobe um servidor TLS com a política do broker e conecta com o mesmo contexto TLS do dispositivo. Casos: dispositivo válido conecta (o CN chega ao broker); certificado de **CA falsa com o mesmo CN** é recusado; cliente sem certificado é recusado; broker sem SAN correspondente ou com hostname errado é recusado pelo dispositivo (proteção contra MITM).

2.6 IaC Security (Dockerfile e Kubernetes)

Prática	Onde	Ameaça mitigada
Build multi-stage; o runtime não tem pip nem compilador	api/Dockerfile	Menor superfície de ataque
Usuário não-root (UID 10001) e código somente leitura	api/ e iot/Dockerfile	Escalada de privilégio
Nenhum segredo na imagem; certificados e chaves montados em runtime	Dockerfiles, compose, k8s	Vazamento pela imagem
runAsNonRoot, readOnlyRootFilesystem, drop ALL, seccomp RuntimeDefault	k8s/api.yaml	Escape de container
requests/limits de CPU e memória	k8s/api.yaml, compose	Negação de serviço
NetworkPolicy default-deny (entrada só do Ingress/Prometheus; saída só DB e DNS)	k8s/api.yaml	Movimento lateral
automountServiceAccountToken: false	k8s/api.yaml	Abuso da API do Kubernetes
Imagem sempre por digest @sha256 e assinada	pipeline + k8s	Troca de imagem
Ingress: TLS 1.2/1.3, 64 KB, 20 rps, /metrics bloqueado	k8s/api.yaml	DoS e exposição de métricas
Compose: portas só em 127.0.0.1, read_only, no-new-privileges, cap_drop ALL	infra/docker-compose.yml	Exposição local

Resultado do Trivy 0.74 (config): 2 Dockerfiles com 0 achados. No manifest K8s, 99 de 100 checagens passaram na primeira execução. O achado KSV-0013 foi corrigido (seção 2.7) e o KSV-0125 (registry fora da lista padrão) ficou como aceite de risco com prazo no .trivyignore.

infra/k8s/api.yaml — contexto de segurança do container

```
allowPrivilegeEscalation: false
readOnlyRootFilesystem: true
privileged: false
capabilities:
  drop: ["ALL"]
resources:
  # evita que um pod esgote o nó (DoS)
  requests: { cpu: 100m, memory: 128Mi }
  limits: { cpu: 500m, memory: 256Mi }
```

2.7 Evidências: testes e correções reais

Categoria	Testes	O que é verificado	Resultado
API · Autenticação / JWT	14	login, expiração, assinatura, alg=none, audience/issuer, adulteração, logout	PASSOU
API · Autorização (RBAC + BOLA)	8	perfis, outra concessionária, 404 em lead alheio, mass assignment	PASSOU

Categoria	Testes	O que é verificado	Resultado
API · Validação e hardening	16	enum/tamanho/log injection, faixas ML, 429, 413, 426, headers, CORS, erro 500, hash obrigatório	PASSOU
API · Criptografia e dados pessoais	8	PII cifrada, CPF mascarado, adulteração, troca de campo, rotação, integridade do modelo	PASSOU
API · Logs de auditoria	3	eventos de login sem segredo no log, 403 auditado, alteração crítica	PASSOU
IoT · Ingestão	23	anti-replay, janela de tempo, esquema, tópico, 4 KB, odômetro, métricas	PASSOU
IoT · Handshake mTLS	5	dispositivo válido, CA falsa, sem certificado, sem SAN, hostname errado	PASSOU

Correções reais encontradas pelos testes, pelas ferramentas e pela revisão

Tudo abaixo foi encontrado e corrigido durante esta Sprint. É a evidência de que o ciclo DevSecOps funciona: a ferramenta ou o teste acha, a correção entra e a verificação passa a garantir.

#	Encontrado por	Problema	Correção
1	Teste (pytest)	Campo note aceitava <code>\r\n</code> (log injection): a regex de controle deixava <code>0x0A</code> e <code>0x0D</code> de fora	Faixa <code>0x0A-0x1F</code> bloqueada (schemas.py); <code>43/44</code> → <code>44/44</code>
2	Semgrep (execução local)	Arquivo de regras era YAML inválido (: sem aspas): no CI a etapa 2 quebraria	Padrões em bloco literal + casos de teste + <code>verificar_regras.py</code> no pipeline
3	Semgrep (casos de teste)	Regra <code>route-without-auth</code> depende <code>ncy</code> não reconhecia parâmetro tipado (<code>p: T = Depends()</code>)	Padrão adicional; 9/9 casos detectados
4	Bandit B101	<code>assert</code> validando <code>MIN_APP_VERSION</code> (some com <code>python -0</code>)	Checagem explícita com <code>ConfigError</code>
5	Trivy KSV-0013	Placeholder de imagem sem tag/digest no manifest K8s	Placeholder <code>@sha256</code> : válido; sed do pipeline ajustado
6	Trivy KSV-0125	Registry (GHCR) fora da lista padrão	Aceite de risco com dono e expiração (31/03/2027)
7	Revisão de versões	Actions e ferramentas defasadas e fixadas por tag (ex.: artefatos do Trivy 0.57 nem existem mais)	Versões atuais, fixadas por SHA de commit
8	Simulação de ataques	Alertas não disparariam: força bruta chegou a só 5 falhas (o rate limit bloqueia antes) contra limiar 30; BOLA chegou a 7 contra 20	Regra soma falhas + bloqueios; limiar de 403 = 5; todos os ataques validados (3.3)
9	Teste mTLS	Certificados sem AKI/SKI são recusados pela verificação X.509 estrita (Python 3.13+)	SKI/AKI explícitos no <code>gerar_certificados.ps1</code>
10	Revisão	Broker sem SAN (verificação de hostname falharia) e sem CRL gerada (broker não subiria)	SAN localhost/mosquitto, CRL e comando de revogação no script
11	Revisão	App mostrava "sessão expirada" para senha errada (401 no login)	401 só limpa sessão se havia token
12	Revisão	Alertas de IoT/mobile usavam métricas que não existem	Serviço de ingestão com métricas reais + métrica de versão do app
13	Revisão	Hash do modelo de ML era opcional mesmo em produção	<code>ConfigError</code> em prod + hash gerado no pipeline
14	Revisão	Log da ingestão IoT era JSON aninhado (filtro do Loki não acharia o evento)	Log plano no mesmo formato da API
15	Semgrep no CI (GitHub Actions)	Scripts de docs/ liam XML com <code>xml.etree</code> (vulnerável a XXE e "XML bomb"); a execução local só cobria <code>api/iot/mobile</code>	defusedxml nos dois scripts; o CI passou a varrer o repositório inteiro
16	Trivy no CI (imagem da API)	Gate da etapa 5 bloqueou a imagem por CVE alta/crítica corrigível: o pip da imagem base ia junto para o runtime	pip removido das imagens da API e do IoT; nova varredura sem achados

Execução no GitHub Actions: o workflow devsecops roda a cada push em

<https://github.com/JoseKretzer/ford-fiap-cyber/actions>. Na execução do commit `a5b4ecd`, as etapas 1 a 5 passaram.

Antes disso, os gates bloquearam dois commits de verdade (itens 15 e 16 acima). As etapas 6 a 8 aparecem como

skipped: o deploy só roda com a variável `DEPLOY_ENABLED`, ativada quando o AKS e os secrets do Azure forem provisionados. Histórico de commits: <https://github.com/JoseKretzer/ford-fiap-cyber/commits/main>.

All checks have passed			×
5 successful and 3 skipped checks			
✓	devsecops / 1 · Secret scanning (Gitleaks) (push)	Successful in 15s	Details
✓	devsecops / 2 · SAST (Semgrep + Bandit) (push)	Successful in 32s	Details
✓	devsecops / 3 · SCA (pip-audit + npm audit + Dependency Review) (push)	Successful in 16s	Details
✓	devsecops / 4 · Testes automatizados (API + IoT) (push)	Successful in 1m	Details
✓	devsecops / 5 · Build, Trivy (imagens + IaC), SBOM e assinatura (push)	Successful in 1m	Details
⌚	devsecops / 6 · Deploy staging (push)	Skipped	Details
⌚	devsecops / 7 · DAST (OWASP ZAP baseline em staging) (push)	Skipped	Details
⌚	devsecops / 8 · Deploy produção (aprovação manual) (push)	Skipped	Details

Figura 3 — Execução real do workflow devsecops no GitHub Actions: etapas 1 a 5 aprovadas; 6 a 8 puladas (deploy desativado).

3. Observabilidade, Monitoramento e Resposta (peso 2,0)

Objetivo: mostrar como o sistema detecta, registra e responde a incidentes. As métricas e os logs abaixo são **reais**. Foram gerados executando a API e a ingestão IoT com observability/simular_trafego.py, que simula 2 horas de uso normal com cinco ataques injetados: força bruta, sondagem BOLA, tokens forjados, app desatualizado e replay de telemetria.

3.1 Plano de monitoramento: logs estruturados

API e ingestão escrevem um evento JSON por linha no stdout. O Grafana Alloy envia os eventos ao **Loki**. Todo evento tem timestamp UTC, nível, nome do evento e, na API, o **request_id**, que liga o log à resposta de erro recebida pelo usuário. O RedactingFilter troca por [REDACTED] campos chamados senha, token, CPF, telefone, e-mail ou VIN.

Evento	Quando é gerado	Campos principais	Na simulação
auth.login.success	Login aceito	user_id, role, dealer_id, client_ip	8
auth.login.failure	Senha errada / usuário inexistente	username, client_ip	13
auth.login.blocked	Rate limit de login disparado	username, scope, client_ip	115
auth.token_rejected	JWT inválido, expirado ou revogado	reason, client_ip, path	25
authz.denied	Perfil sem permissão ou fora do escopo	user_id, role, permission, target_dealer	21
lead.status_changed	Alteração crítica de dado de negócio	user_id, lead_id, old_status, new_status	24
iot.telemetry.rejected	Telemetria recusada pela ingestão	reason, device_id	26
http.access	Toda requisição (inclui 426 do app antigo)	method, route, status, duration_ms, user_id	718
app.error	Exceção não tratada (500)	exc_type, stack (só no log)	0

Retenção: eventos de auditoria ficam **1 ano** em armazenamento somente-anexo. Logs de acesso ficam **6 meses**, o mínimo exigido pelo Marco Civil da Internet (art. 15).

Exemplos reais de log (extraídos de observability/exemplos_logs.jsonl)

```
{
  "ts": "2026-09-25T15:45:43.412+00:00",
  "level": "INFO",
  "logger": "vinshare.audit",
  "msg": "auth.login.failure",
  "event": "auth.login.failure",
  "request_id": "e46816d8c34b4628a85623e7e96d4b10",
  "client_ip": "203.0.113.50",
  "method": "POST",
  "path": "/api/v1/auth/login",
  "username": "gestor.sp01"
}
{"ts": "2026-09-25T15:45:44.217+00:00",
  "level": "INFO",
  "logger": "vinshare.audit",
  "msg": "auth.login.blocked",
  "event": "auth.login.blocked",
  "request_id": "a162d10b8446495685e2d6609b676382",
  "client_ip": "203.0.113.50",
  "method": "POST",
  "path": "/api/v1/auth/login",
  "username": "gestor.sp01",
  "scope": "login_user"
}
{"ts": "2026-09-25T15:45:46.297+00:00",
  "level": "INFO",
  "logger": "vinshare.audit",
  "msg": "authz.denied",
  "event": "authz.denied",
  "request_id": "e3e1b25cb65a4c0c9d05ea2fe3d8e0d7",
  "client_ip": "10.2.7.12",
  "method": "GET",
  "path": "/api/v1/leads/L-1001",
  "user_id": "u-200",
  "role": "consultor",
  "permission": "dealer_scope",
  "target_dealer": "SP01"
}
{"ts": "2026-09-25T15:45:47.390+00:00",
  "level": "INFO",
  "logger": "vinshare.audit",
  "msg": "auth.token_rejected",
  "event": "auth.token_rejected",
  "request_id": "d7658ef9124d40f0bbd8a61c4c2a7a1e",
  "client_ip": "203.0.113.50",
  "method": "GET",
  "path": "/api/v1/admin/audit-events",
  "reason": "invalid_token"
}
{"ts": "2026-09-25T15:45:42.115+00:00",
  "level": "INFO",
  "logger": "vinshare.audit",
  "msg": "lead.status_changed",
  "event": "lead.status_changed",
  "request_id": "d18fef85588544f6b922299e6f61cc33",
  "client_ip": "10.1.4.21",
  "method": "PATCH",
  "path": "/api/v1/leads/L-1003",
  "user_id": "u-100",
  "lead_id": "L-1003",
  "dealer_id": "SP01",
  "old_status": "novo",
  "new_status": "agendado"
}
{"ts": "2026-09-25T12:45:47.0300",
  "level": "WARNING",
  "logger": "vinshare.iot",
  "event": "iot.telemetry.rejected",
  "reason": "replay",
  "device_id": "veh-00000000000000a1"
}
{"ts": "2026-09-25T15:45:42.867+00:00",
  "level": "INFO",
  "logger": "vinshare.api",
  "msg": "http_request",
  "event": "http.access",
  "request_id": "624db6ed0ea94793a6cbaf768f55ff56",
  "client_ip": "10.1.4.77",
  "method": "GET",
  "route": "unmatched",
  "status": 426,
  "duration_ms": 0.07,
  "user_id": null
}
```

3.2 Métricas e alertas (API, mobile, IoT e ML)

A API e a ingestão expõem métricas Prometheus, acessíveis só pela rede interna. As regras ficam em observability/alert_rules.yml (métricas) e observability/loki-rules/iot.yml (logs do broker). O Alertmanager envia *warning* ao canal do time no Teams e *critical* ao plantão de segurança. As URLs de webhook ficam em arquivos de segredo.

Área	Fonte / métrica	Alerta	Condição	Sev.	Playbook
API	auth_login_total{failure} + rate_limited_total{login}	ForcaBrutaLogin	> 30 em 5 min	crítico	PB-01
API	vinshare_authz_denied_total	SondagemDeAutorizacao	> 5 em 5 min	crítico	PB-02
API	vinshare_auth_token_rejected_total	TokensForjadosOuExpirados	> 20 inválidos em 5 min	crítico	PB-03
API	vinshare_http_requests_total	ErrosServidor	5xx > 2% por 5 min	crítico	—

Área	Fonte / métrica	Alerta	Condição	Sev.	Playbook
API	vinshare_http_request_duration_seconds	LatenciaAlta	p95 > 500 ms por 10 min	aviso	—
Mobile	vinshare_mobile_requests_total{bloqueado}	AppDesatualizadoEmUso	> 10 (426) em 15 min	aviso	—
IoT	vinshare_iot_messages_total{#aceita}	TelemetriaRejeitada	> 10 em 10 min	crítico	PB-04
IoT	vinshare_iot_last_accepted_timestamp	TelemetriaParou	nada aceito há 15 min	aviso	—
IoT	Log do broker (LogQL)	FalhaAutenticacaoMQTT	> 20 recusadas TLS em 10 min	crítico	PB-04
ML	vinshare_ml_churn_score (histograma)	DriftScoreChurn	> 50% com risco alto em 1 h	aviso	PB-05
ML	vinshare_ml_predictions_total	VolumePredicoesAnomalo	> 500 em 5 min	aviso	PB-05

Mobile (crashes): a estabilidade do app é monitorada no Firebase Crashlytics, com alerta nativo no console quando os usuários sem crash ficam abaixo de 99%. Essa métrica não passa pelo Prometheus.

3.3 Dashboard e validação dos alertas

O dashboard observability/grafana/vinshare-security.json é provisionado automaticamente no Grafana pelo docker-compose, com Prometheus e Loki como fontes. A Figura 4 mostra os mesmos painéis, montados com as métricas coletadas na simulação. O último painel mostra quando cada regra de alerta teria disparado.

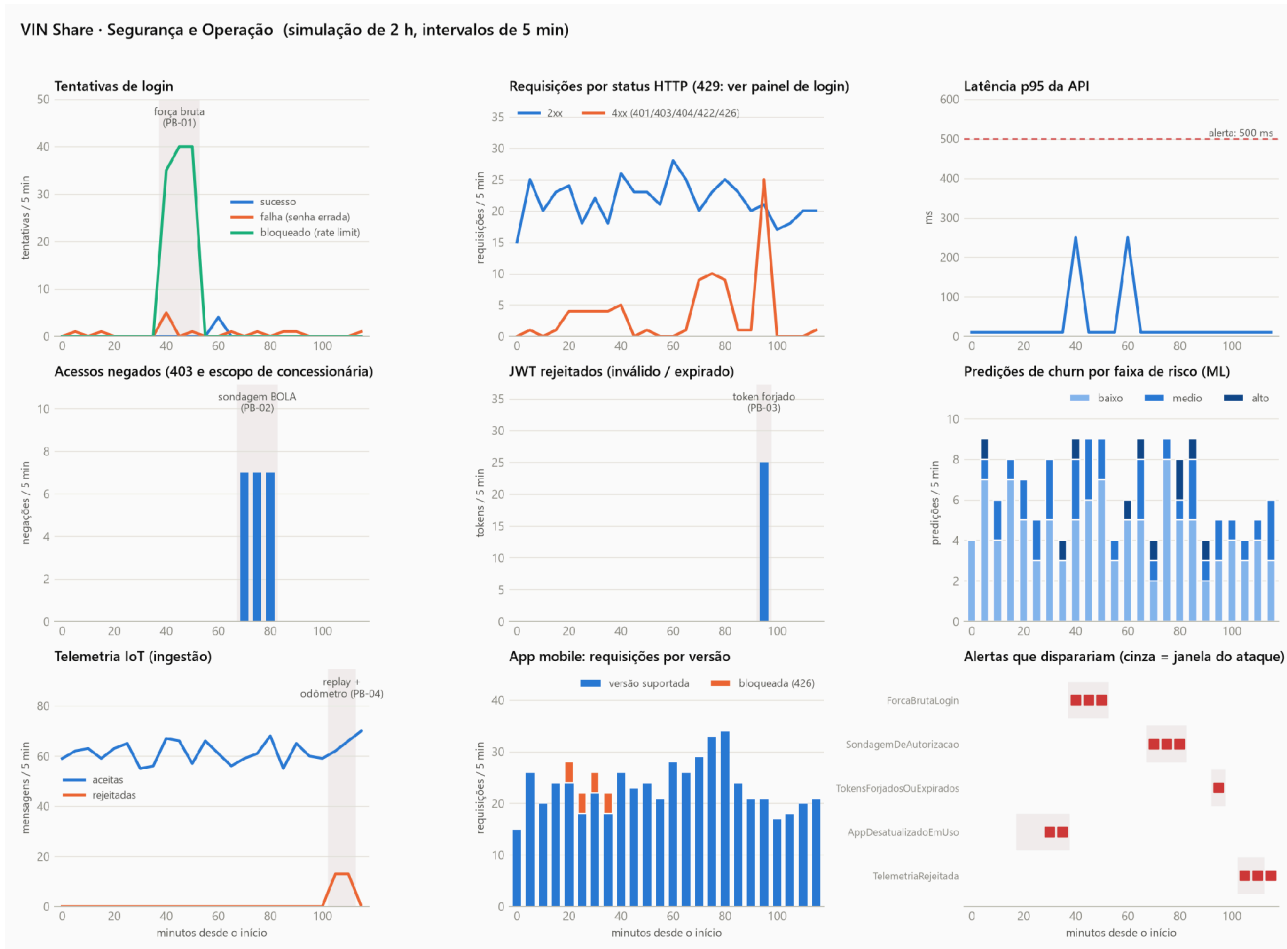


Figura 4 — Painel de segurança e operação com as métricas reais da simulação (2 h, intervalos de 5 min).

Validação dos limiares (observability/avaliar_alertas.py): cada regra foi avaliada sobre a série da simulação. Critério: todo ataque dispara o alerta do seu playbook e o tráfego normal não dispara nada.

Alerta	Limiar	Pico observado	Ataque (min)	Disparou em (min)	Resultado
ForcaBrutaLogin	falhas + bloqueios de login > 30 em 5 min	41	40, 45, 50	40, 45, 50	detectou
SondagemDeAutorizacao	negações 403 > 5 em 5 min	7	70, 75, 80	70, 75, 80	detectou

Alerta	Limiar	Pico observado	Ataque (min)	Disparou em (min)	Resultado
TokensForjadosOuExpirados	JWT inválidos > 20 em 5 min	25	95	95	detectou
ErrosServidor	5xx > 2% em 5 min	0	—	—	sem disparo (correto)
LatenciaAlta	p95 > 0,5 s	0.25	—	—	sem disparo (correto)
AppDesatualizadoEmUso	requisições 426 > 10 em 15 min	12	20, 25, 30, 35	30, 35	detectou
TelemetriaRejeitada	telemetria rejeitada > 10 em 10 min	26	105, 110	105, 110, 115	detectou
DriftScoreChurn	> 50% das predições com score > 0,7 em 1 h	0.107	—	—	sem disparo (correto)
VolumePredicoesAnomalo	predições > 500 em 5 min	9	—	—	sem disparo (correto)

3.4 Plano de resposta a incidentes

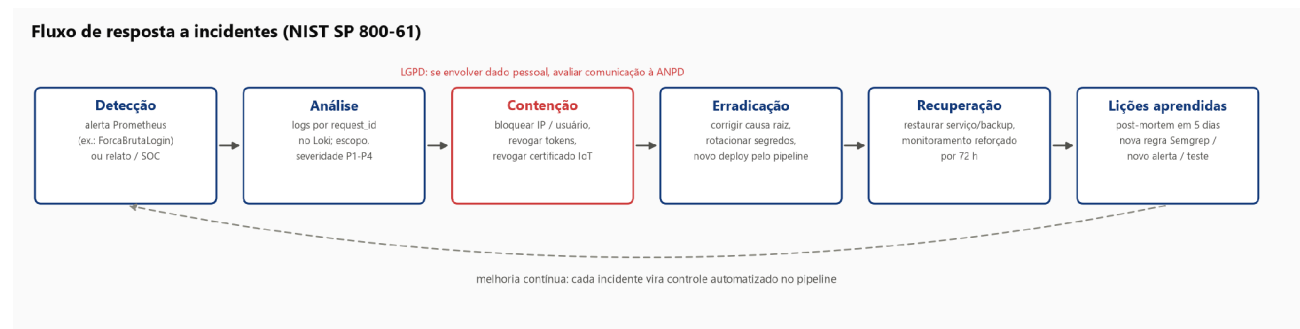


Figura 5 — Fluxo de resposta a incidentes, baseado no NIST SP 800-61.

Classificação de severidade

Sev.	Exemplo	1ª resposta	Contenção	Quem é acionado
P1	Vazamento de dados pessoais; chave comprometida	15 min	até 4 h	Plantão, líder de segurança, DPO, jurídico
P2	Ataque ativo (força bruta, BOLA, replay IoT) já contido pelos controles	1 h	até 24 h	Plantão e líder técnico
P3	Vulnerabilidade alta sem exploração; alerta de drift	1 dia útil	até 7 dias	Time do produto
P4	Achado baixo, melhoria	5 dias úteis	próxima sprint	Time do produto

Playbooks

ID	Cenário	Detecção → Análise	Contenção	Erradicação → Recuperação
PB-01	Força bruta / credential stuffing	ForçaBrutaLogin → IPs e usuários em auth.login.failure/blocked	Rate limit já bloqueia; bloquear IP/ASN no Ingress/WAF; forçar troca de senha das contas-alvo	Avaliar MFA para gestores; monitorar 72 h; desbloquear contas verificadas
PB-02	Sondagem BOLA / abuso interno	SondagemDeAutorizacao → user_id e target_dealer em authz.denied	Desativar usuário e revogar tokens (jti)	Revisar permissões da concessionária; conferir se houve 200 indevido; comunicar gestor/RH
PB-03	Token forjado / JWT_SECRET vazado	TokensForjadosOuExpirados → padrão de tokens e IPs	Rotacionar JWT_SECRET no Key Vault (invalida todos os tokens)	Achar a origem do vazamento (Gitleaks, logs); redeploy; novo login para todos
PB-04	Dispositivo IoT comprometido / replay	TelemetriaRejeitada ou FalhaAutenticacaoMQTT → device_id e reason	gerar_certificados.ps1 -Revogar <id> (CRL); descartar telemetria do período	Reemitir certificado; investigar o firmware; reprocessar dados válidos
PB-05	Abuso ou drift do modelo de ML	DriftScoreChurn / VolumePredicoesAnomalo	Rate limit por usuário nas predições; congelar a versão do modelo	Investigar envenenamento; retreinar com dados validados; publicar novo hash
PB-06	Vazamento de dados pessoais	Qualquer fonte (alerta, relato, terceiro)	Isolar o componente; preservar evidências (logs, snapshots)	Avaliar risco aos titulares com o DPO; comunicar ANPD e titulares (LGPD art. 48)

LGPD: incidente com dado pessoal que possa gerar risco ou dano relevante aos titulares deve ser comunicado à ANPD e aos titulares (art. 48). O Regulamento de Comunicação de Incidente de Segurança (Resolução CD/ANPD nº 15/2024) fixa o prazo de **3 dias úteis**. O playbook PB-06 aciona o DPO já na fase de análise.

4. Compliance, Riscos e Segurança Contínua (peso 2,5)

4.1 Revisão final dos riscos (STRIDE)

Ameaças por componente, com o controle implementado, a verificação contínua no pipeline e o risco residual.

ID	Comp.	STRIDE	Ameaça	Controle implementado	Verificação contínua	Residual
S-01	API	S	Força bruta / credential stuffing no login	bcrypt 12, rate limit duplo, mensagem genérica, alerta	Etapa 4 (testes), alerta validado	Baixo
S-02	API	S	JWT forjado (alg=none, chave errada, claims)	HS256 fixo, iss/aud/exp/jti obrigatórios	Etapa 4, Semgrep (2 regras)	Baixo
S-03	IoT	S	Dispositivo falso publicando telemetria	mTLS, certificado por veículo, CRL, ACL	Etapa 4 (teste mTLS)	Baixo
T-01	IoT	T	Telemetria adulterada ou repetida (replay)	TLS; ingestão com janela de tempo, msg_id e odômetro	Etapa 4 (23 testes), alerta	Baixo
T-02	API	T	Mass assignment (enviar dealer_id/role)	extra=forbid nos schemas	Etapa 4 (testes)	Baixo
T-03	ML	T	Modelo trocado ou envenenado	JSON + SHA-256 obrigatório em prod; alerta de drift	Etapas 4 e 5, Semgrep (pickle)	Médio
T-04	CI/CD	T	Dependência, imagem ou Action maliciosa	SCA, SBOM, cosign, pin por SHA	Etapas 3 e 5	Médio
R-01	API	R	Usuário nega ter alterado um lead	Log de auditoria com user_id e request_id (1 ano)	Etapa 4 (testes)	Baixo
I-01	Dados	I	Vazamento do banco ou de backup	AES-256-GCM por campo, TDE, backup cifrado	Etapa 4 (testes)	Baixo
I-02	API	I	BOLA: ler leads de outra concessionária	Escopo por concessionária, 404	Etapa 4, alerta validado	Baixo
I-03	Repo	I	Segredo commitado	Key Vault + env; Gitleaks em todo o histórico	Etapa 1	Baixo
I-04	API	I	Stack trace ou dado sensível no erro/log	problem+json, RedactingFilter	Etapa 4, Semgrep	Baixo
I-05	Mobile	I	Token extraído do aparelho	SecureStore, allowBackup false, HTTPS obrigatório	Semgrep (AsyncStorage)	Médio
I-06	IoT	I	Telemetria interceptada na rede	TLS obrigatório, porta 1883 inexistente	Semgrep (MQTT sem TLS)	Baixo
D-01	API	D	Flood de requisições / payload grande	Rate limit, 64 KB, limit-rps, limites K8s	Etapa 4, DAST	Médio
D-02	IoT	D	Flood MQTT	Pacote de 4 KB, limites de fila, ACL	Revisão de config	Médio
E-01	API	E	Consultor acessando função de admin	RBAC no servidor por permissão	Etapa 4, Semgrep (rota sem auth)	Baixo
E-02	Infra	E	Escape de container / escalada no cluster	Não-root, drop ALL, read-only, seccomp, sem SA token	Etapa 5 (Trivy config)	Baixo

Riscos residuais médios (aceitos para esta fase, com plano): T-03 inclui envenenamento dos dados de treino; T-04 cobre supply chain, que nunca chega a zero; I-05 considera aparelhos com root/jailbreak (plano: detecção de root e certificate pinning no build nativo); D-01 e D-02 dependem de proteção DDoS de rede (Azure DDoS Protection / WAF).

4.2 Riscos do ciclo DevSecOps

ID	Risco	Controle	Residual
DS-01	PR de fork rouba segredos do pipeline	Evento pull_request (sem segredos); nenhum pull_request_target	Baixo
DS-02	Credencial de nuvem vazada do CI	OIDC com token de curta duração; nenhum segredo de nuvem guardado	Baixo
DS-03	Imagem adulterada entre build e deploy	Deploy por digest + cosign verify com identidade do workflow	Baixo

ID	Risco	Controle	Residual
DS-04	Bypass dos gates (merge direto / admin)	Branch protection para todos (inclui admins) + Environment com revisor	Baixo
DS-05	Action de terceiro comprometida	Pin por SHA, permissões mínimas, Dependabot	Médio
DS-06	Fadiga de alertas / falso negativo das ferramentas	Regras próprias testadas, limiares validados por simulação, triagem documentada	Médio
DS-07	Exceção de segurança esquecida	.trivyignore com dono e data de expiração	Baixo

4.3 Mapeamento OWASP ASVS 4.0.3

Req.	Descrição resumida	Como atendemos	Status
2.1.1	Senhas com 12+ caracteres	LoginRequest.min_length=12	Implementado e testado
2.2.1	Controles anti-automação	Rate limit por usuário+IP e por IP	Implementado e testado
2.4.1	Hash de senha resistente (bcrypt/argon2...)	bcrypt custo 12	Implementado
3.3.1	Logout invalida a sessão	Revogação do jti	Implementado e testado
3.5.3	Tokens stateless assinados e protegidos contra adulteração	HS256 fixo, claims obrigatórios, sem alg=none	Implementado e testado
4.1.1 / 4.1.3	Controle de acesso no servidor, com menor privilégio	RBAC em dependências FastAPI	Implementado e testado
4.2.1	Proteção contra IDOR	Escopo por concessionária	Implementado e testado
5.1.2 / 5.1.3	Proteção contra mass assignment; validação por allow-list	Pydantic extra=forbid, enums, faixas	Implementado e testado
6.2.1 / 6.2.2	Criptografia falha de forma segura, com algoritmos aprovados	AES-256-GCM, DecryptionError	Implementado e testado
6.4.1	Gestão de chaves	Azure Key Vault, ID de chave para rotação	Implementado (código) / Key Vault planejado
7.1.1	Sem credenciais ou tokens em log	RedactingFilter + teste	Implementado e testado
7.1.3	Registrar eventos de segurança	auth., authz.denied, lead.status_changed, iot.	Implementado e testado
7.4.1	Erro genérico com ID para o suporte	problem+json com request_id	Implementado e testado
8.3.1	Dado sensível no corpo/cabeçalho, nunca na URL	Login por POST no corpo; token no header	Implementado
9.1.1 / 9.1.2	TLS em todas as conexões, cifras fortes	Ingress TLS 1.2/1.3, MQTT 8883 (mTLS testado)	Implementado e testado (IoT)
14.2.1	Componentes atualizados, sem CVE conhecida	Dependabot + pip-audit + Trivy (0 CVEs hoje)	Executado
14.3.2	Modos de debug desligados em produção	Swagger/OpenAPI off em APP_ENV=prod	Implementado
14.4.3-14.4.7	Cabeçalhos de segurança	CSP, nosniff, HSTS, frame-ancestors	Implementado e testado

4.4 OWASP API Security Top 10 (2023)

Risco	Controle no VIN Share	Status
API1 Broken Object Level Authorization	Escopo de concessionária em todo recurso; 404 para recurso alheio	Implementado e testado
API2 Broken Authentication	JWT de 15 min com revogação; bcrypt; rate limit no login	Implementado e testado
API3 Broken Object Property Level Authorization	extra=forbid na entrada; CPF mascarado na saída	Implementado e testado
API4 Unrestricted Resource Consumption	Rate limit, corpo de 64 KB, limites K8s e Ingress, limit ≤ 500	Implementado e testado
API5 Broken Function Level Authorization	Permissão por função (audit:read só admin)	Implementado e testado
API6 Unrestricted Access to Sensitive Business Flows	Alerta de volume de predições (extração do modelo); rate limit	Implementado
API7 Server Side Request Forgery	A API não busca URLs vindas do usuário; egress restrito por NetworkPolicy	Não aplicável por design
API8 Security Misconfiguration	Headers, CORS allow-list, config que falha no startup, Trivy config	Implementado e testado
API9 Improper Inventory Management	Versão /api/v1; docs off em prod; staging separado	Implementado
API10 Unsafe Consumption of APIs	Telemetria tratada como não confiável (esquema, tempo, anti-replay)	Implementado e testado

4.5 OWASP Mobile Top 10 (2024)

Risco	Controle no app VIN Share	Status
M1 Improper Credential Usage	Nenhuma credencial no app; só o token do usuário no SecureStore	Implementado (código)
M2 Inadequate Supply Chain Security	npm audit + Dependabot npm + Dependency Review	Configurado no pipeline
M3 Insecure Authentication/Authorization	Autorização só na API; 401 com token limpa a sessão	Implementado (código)
M4 Insufficient Input/Output Validation	Validação na API (allow-list); o app não monta HTML com dados da API	Implementado
M5 Insecure Communication	Somente HTTPS; usesCleartextTraffic=false; pinning no build nativo	Implementado / pinning planejado
M6 Inadequate Privacy Controls	Sem localização/contatos; CPF mascarado; nada de PII em log	Implementado (código)
M7 Insufficient Binary Protections	ProGuard/R8 e shrink de recursos no release	Configurado
M8 Security Misconfiguration	allowBackup=false; permissões bloqueadas; versão mínima forçada (426)	Implementado e testado (API)
M9 Insecure Data Storage	expo-secure-store; regra Semgrep contra AsyncStorage (testada)	Implementado
M10 Insufficient Cryptography	Criptografia no servidor com AES-GCM; o app não implementa cripto própria	Implementado

4.6 LGPD: dados pessoais, telemetria e localização

Artigo	Exigência	Aplicação no VIN Share	Status
Art. 6º, I-III	Finalidade, adequação e necessidade	Telemetria limitada a odômetro, códigos de falha e vida do óleo (esquema estrito rejeita campos extras); CPF sempre mascarado	Implementado e testado
Art. 7º	Base legal	Execução de contrato/garantia para manutenção; legítimo interesse (com LIA e opt-out) para lembretes; consentimento para localização e ofertas	Planejado (texto jurídico)
Art. 9º	Transparência	Aviso de privacidade no app, explicando telemetria e modelo de churn	Planejado
Art. 13, §4º	Pseudonimização	VIN pseudonimizado (HMAC) no IoT, analytics e ML; continua tratado como dado pessoal	Implementado
Art. 16	Eliminação ao fim do tratamento	Telemetria bruta: 24 meses; auditoria: 1 ano; acesso: 6 meses	Planejado (job de expurgo)
Art. 18	Direitos do titular	Fluxos de acesso, correção, eliminação e revogação de consentimento pelo app/SAC	Planejado
Art. 37 / 38	Registro das operações e RIPD	Inventário de dados + RIPD, pois telemetria veicular é tratamento de risco elevado	Planejado
Art. 46	Medidas de segurança	AES-256-GCM, TLS/mTLS, RBAC, logs, pipeline DevSecOps	Implementado
Art. 48	Comunicação de incidente	Playbook PB-06; ANPD em 3 dias úteis (Res. CD/ANPD nº 15/2024)	Implementado (processo)

Localização: não é dado sensível pela LGPD (art. 5º, II), mas revela hábitos e endereço do titular. Por isso a solução **não coleta GPS**, e o esquema da ingestão rejeita qualquer campo extra. Se uma funcionalidade futura precisar de localização (ex.: concessionária mais próxima), a coleta será pontual, com consentimento e precisão reduzida (cidade/UF).

4.7 Plano de segurança contínua

Rotina	Frequência	Como / ferramenta	Responsável	Evidência
Revisão de dependências	A cada PR + semanal	Dependabot (pip, npm, Docker, compose, Actions), pip-audit, npm audit, Trivy. SLA: crítica 7 dias, alta 30, média 90	Tech lead	PRs do Dependabot; aba Security
Testes de segurança	A cada PR	Gitleaks, Semgrep (+ teste das regras), Bandit, pytest API + IoT	Todo o time	Checks do PR
	Semanal / a cada deploy	Re-scan agendado; ZAP baseline em staging; revisão	DevOps	Artefatos do workflow

Rotina	Frequência	Como / ferramenta	Responsável	Evidência
		das exceções do .trivyignore		
	A cada mudança de alerta	Reexecutar simulação + avaliar_alertas.py (todo ataque dispara, nada de falso positivo)	Segurança	Saída do avaliador
	Semestral	Pentest (API, app e broker) + revisão STRIDE de novas features	Segurança	Relatório de pentest
Auditoria de permissões	Mensal	Usuários ativos x concessionárias/RH; desligados removidos em 24 h; revisão de authz.denied	Gestor + admin	Planilha de revisão assinada
	Trimestral	Acessos da org GitHub, Azure RBAC, Key Vault e ACL do broker	Segurança	Export de acessos
Rotação de segredos	90 dias / anual	JWT_SECRET e certificados IoT a cada 90 dias; DATA_KEY anual (recifra via ID de chave)	DevOps	Histórico de versões do Key Vault
Backup e recuperação (continuação da rotina iniciada na Fase 3)	Diário	Backup automático do PostgreSQL + PITR de 7 dias, cópia geo-redundante cifrada, retenção de 35 dias	DevOps	Relatório do Azure Backup
	Mensal	Teste de restore em ambiente isolado. Meta: RPO ≤ 24 h (PITR ~5 min), RTO ≤ 4 h	DevOps	Ata do teste com tempo medido
	Contínuo	Key Vault com soft-delete e purge protection (sem a chave, o backup cifrado se perde)	Segurança	Configuração do Key Vault

4.8 Checklist de conformidade

Legenda: ✓ **Verificado** (implementado e executado/testado nesta Sprint) · ① **Configurado** (arquivo pronto; roda quando o repositório for publicado e a infraestrutura provisionada) · ○ **Planejado**.

Área	Item	Status	Evidência
Pipeline	Secret scanning em todo o histórico, com regras próprias	✓ / ①	1.5 (execução local) · devsecops.yml
Pipeline	SAST com regras próprias testadas	✓ / ①	1.5 · .semgrep/
Pipeline	SCA + Dependabot	✓ / ①	pip-audit, Trivy fs · dependabot.yml
Pipeline	Container + IaC scan, SBOM, assinatura	✓ / ①	Trivy config executado · devsecops.yml (etapa 5)
Pipeline	Actions fixadas por SHA, OIDC, permissões mínimas	①	devsecops.yml
Pipeline	DAST em staging e aprovação para produção	①	devsecops.yml (etapas 7-8)
Código	Criptografia AES-256-GCM de PII + rotação de chave	✓	api/app/crypto.py · TestDataProtection
Código	JWT seguro (alg fixo, claims, expiração, revogação)	✓	api/app/security.py · TestAuthentication
Código	Rate limit, limite de corpo e validação por allow-list	✓	api/app/ · TestHardening
Código	RBAC com 3 perfis + proteção BOLA	✓	api/app/security.py · TestAuthorization
Código	Erros padronizados sem vazamento	✓	api/app/errors.py · TestHardening
ML	Integridade do modelo (hash obrigatório em prod)	✓	api/app/model.py, config.py · TestDataProtection/TestHardening
Mobile	Token em armazenamento seguro, só HTTPS, versão mínima	✓ (API) / ① (app)	secureSession.ts · teste 426
Mobile	Certificate pinning e detecção de root	○	Build nativo (EAS) - próxima fase
IoT	TLS 1.2+, mTLS, ACL e CRL	✓ (mTLS) / ① (broker)	iot/mosquitto/ · test_mtls
IoT	Ingestão com anti-replay, esquema e janela de tempo	✓	iot/ingestion.py · test_ingestion
Infra	Dockerfiles não-root, multi-stage, sem segredos	✓	Trivy config: 0 achados
Infra	K8s hardening + NetworkPolicy	✓	Trivy config: 0 achados (1 aceite com prazo)

Área	Item	Status	Evidência
Observab.	Logs JSON com auditoria e redação de PII	✓	api/app/logging_setup.py · TestAuditLogging
Observab.	Métricas + 11 alertas (API, mobile, IoT, ML)	✓	alert_rules.yml · avaliar_alertas.py
Observab.	Dashboard Grafana + Loki + Alertmanager	◦	observability/, infra/docker-compose.yml
Observab.	Plano de resposta com 6 playbooks	✓	Seção 3.4
Compliance	STRIDE + riscos DevSecOps + OWASP ASVS/API/Mobile	✓	Seções 4.1 a 4.5
LGPD	Minimização, pseudonimização, cifra, incidente	✓	Seção 4.6
LGPD	RIPD, inventário (ROPA), direitos do titular	◦	Jurídico/DPO + backlog
Contínuo	Rotinas de dependências, testes, permissões e backup	✓ (definido)	Seção 4.7

Anexo A. Estrutura do repositório e como executar

```

ford-fiap-cyber/
├── .github/workflows/devsecops.yml    pipeline DevSecOps (8 etapas, Actions fixadas por SHA)
├── .github/dependabot.yml            SCA contínuo (pip, npm, Docker, compose, Actions)
├── .gitignore                        regras e exceções das ferramentas
├── .gitleaks.toml .semgrep/ .zap/    .trivyignore
├── api/                             API FastAPI (app/, tests/, Dockerfile)
├── mobile/                          sessão segura + hardening Android (Expo)
├── iot/                             broker (TLS/mTLS/ACL/CRL), ingestão, cliente TLS, PKI, tests/
├── infra/                           docker-compose (stack completa), k8s/api.yaml, .env.example
├── observability/                  Prometheus, Alertmanager, Loki, Alloy, Grafana, simulador, avaliador
└── docs/                          este documento, figuras, evidências e geradores

```

Comandos (PowerShell, a partir da raiz do repositório)

```

# 1) Ambiente e testes
python -m venv .venv
.venv\Scripts\pip install -r api\requirements-dev.txt -r iot\requirements.txt
cd api; ..\venv\Scripts\python -m pytest -v; cd ../iot; ..\venv\Scripts\python -m pytest -v; cd ..

# 2) Ferramentas de segurança (as mesmas do pipeline)
.venv\Scripts\pip install semgrep bandit pip-audit
.venv\Scripts\python .semgrep\verificar_regras.py
.venv\Scripts\semgrep scan --config .semgrep\vinshare-rules.yml --config p/owasp-top-ten --exclude .semgrep api iot mobile
.venv\Scripts\bandit -r api/app iot -x iot/tests -ll -ii
.venv\Scripts\pip-audit -r api\requirements.txt -r iot\requirements.txt

# 3) Simulação de ataques, validação dos alertas, figuras e documento
.venv\Scripts\python observability\simular_trafego.py
.venv\Scripts\python observability\avaliar_alertas.py
.venv\Scripts\python docs\gerar_figuras.py; .venv\Scripts\python docs\gerar_documento.py

```

Anexo B. Lista completa dos testes automatizados

#	Categoria	Teste	Resultado
1	API · Autenticação / JWT	test_health_is_public	PASSOU
2	API · Autenticação / JWT	test_login_success_returns_short_live_d_jwt	PASSOU
3	API · Autenticação / JWT	test_login_failure_is_generic[gestor.sp01]	PASSOU
4	API · Autenticação / JWT	test_login_failure_is_generic[usuario.i nexistente]	PASSOU
5	API · Autenticação / JWT	test_protected_endpoint_without_tok en	PASSOU
6	API · Autenticação / JWT	test_rejects_bad_tokens[expirado]	PASSOU
7	API · Autenticação / JWT	test_rejects_bad_tokens[assinatura_e rrada]	PASSOU
8	API · Autenticação / JWT	test_rejects_bad_tokens[audience_err ada]	PASSOU
9	API · Autenticação / JWT	test_rejects_bad_tokens[issuer_errad o]	PASSOU
10	API · Autenticação / JWT	test_rejects_bad_tokens[perfil_invalid o]	PASSOU
11	API · Autenticação / JWT	test_rejects_bad_tokens[alg_none]	PASSOU
12	API · Autenticação / JWT	test_rejects_token_missing_required_ claim	PASSOU
13	API · Autenticação / JWT	test_tampered_payload_is_rejected	PASSOU
14	API · Autenticação / JWT	test_logout_revokes_token	PASSOU
15	API · Autorização (RBAC + BOLA)	test_consultor_cannot_read_audit	PASSOU
16	API · Autorização (RBAC + BOLA)	test_consultor_cannot_read_service_s hare	PASSOU
17	API · Autorização (RBAC + BOLA)	test_gestor_reads_own_dealer_servic e_share	PASSOU
18	API · Autorização (RBAC + BOLA)	test_gestor_cannot_read_other_deale r	PASSOU
19	API · Autorização (RBAC + BOLA)	test_bola_lead_of_other_dealer_retur ns_404	PASSOU
20	API · Autorização (RBAC + BOLA)	test_leads_are_scoped_to_dealer	PASSOU
21	API · Autorização (RBAC + BOLA)	test_admin_accesses_any_dealer	PASSOU
22	API · Autorização (RBAC + BOLA)	test_mass_assignment_is_blocked	PASSOU
23	API · Validação e hardening	test_valid_lead_update	PASSOU
24	API · Validação e hardening	test_invalid_input_rejected[enum_inv alido]	PASSOU
25	API · Validação e hardening	test_invalid_input_rejected[texto_lon go]	PASSOU
26	API · Validação e hardening	test_invalid_input_rejected[log_injecti on]	PASSOU
27	API · Validação e hardening	test_ml_input_bounds	PASSOU
28	API · Validação e hardening	test_ml_prediction_ok	PASSOU
29	API · Validação e hardening	test_login_rate_limit	PASSOU
30	API · Validação e hardening	test_payload_too_large	PASSOU
31	API · Validação e hardening	test_security_headers	PASSOU
32	API · Validação e hardening	test_cors_blocks_unknown_origin	PASSOU
33	API · Validação e hardening	test_minimum_app_version[versao_a ntiga]	PASSOU
34	API · Validação e hardening	test_minimum_app_version[versao_m inima]	PASSOU
35	API · Validação e hardening	test_minimum_app_version[versao_n ova]	PASSOU
36	API · Validação e hardening	test_minimum_app_version[versao_in valida]	PASSOU
37	API · Validação e hardening	test_prod_requires_model_hash	PASSOU
38	API · Validação e hardening	test_unhandled_error_does_not_leak	PASSOU
39	API · Criptografia e dados pessoais	test_pii_encrypted_at_rest	PASSOU
40	API · Criptografia e dados pessoais	test_cpf_masked_in_api	PASSOU
41	API · Criptografia e dados pessoais	test_ciphertext_tampering_detected	PASSOU
42	API · Criptografia e dados pessoais	test_ciphertext_cannot_be_moved_b etween_fields	PASSOU
43	API · Criptografia e dados pessoais	test_key_rotation_reads_old_data	PASSOU
44	API · Criptografia e dados pessoais	test_blind_index_search	PASSOU
45	API · Criptografia e dados pessoais	test_mask_cpf	PASSOU
46	API · Criptografia e dados pessoais	test_model_integrity_check	PASSOU
47	API · Logs de auditoria	test_login_events_logged_without_se crets	PASSOU
48	API · Logs de auditoria	test_authz_denied_is_audited	PASSOU
49	API · Logs de auditoria	test_critical_change_is_audited	PASSOU
50	IoT · Ingestão	test_valid_message_accepted	PASSOU
51	IoT · Ingestão	test_replay_rejected	PASSOU
52	IoT · Ingestão	test_same_msg_id_other_device_is_n ot_replay	PASSOU

#	Categoria	Teste	Resultado
53	IoT · Ingestão	test_time_window[1789999699-mensagem_antiga]	PASSOU
54	IoT · Ingestão	test_time_window[179000031-times_tamp_futuro]	PASSOU
55	IoT · Ingestão	test_invalid_topic[vehicles/veh-3f9a1c2b7d0e4a18/commands]	PASSOU
56	IoT · Ingestão	test_invalid_topic[vehicles/+/telemetry]	PASSOU
57	IoT · Ingestão	test_invalid_topic[vehicles/VIN9BFZH54P0R8123456/telemetry]	PASSOU
58	IoT · Ingestão	test_invalid_topic[vehicles/veh-3f9a1c2b7d0e4a18/telemetry/extra]	PASSOU
59	IoT · Ingestão	test_schema_violations[campo_extra]	PASSOU
60	IoT · Ingestão	test_schema_violations[odometro_negativo]	PASSOU
61	IoT · Ingestão	test_schema_violations[tipo_errado]	PASSOU
62	IoT · Ingestão	test_schema_violations[oleo_fora_faixa]	PASSOU
63	IoT · Ingestão	test_schema_violations[dtc_injecao]	PASSOU
64	IoT · Ingestão	test_schema_violations[dtc_demais]	PASSOU
65	IoT · Ingestão	test_schema_violations[msg_id_invalido]	PASSOU
66	IoT · Ingestão	test_schema_violations[ts_float]	PASSOU
67	IoT · Ingestão	test_schema_violations[nao_objeto]	PASSOU
68	IoT · Ingestão	test_invalid_json	PASSOU
69	IoT · Ingestão	test_oversized_payload	PASSOU
70	IoT · Ingestão	test_odometer_regression_flagged	PASSOU
71	IoT · Ingestão	test_replay_cache_expires	PASSOU
72	IoT · Ingestão	test_metrics_count_each_result	PASSOU
73	IoT · Handshake mTLS	test_device_with_valid_certificate_connects	PASSOU
74	IoT · Handshake mTLS	test_certificate_from_rogue_ca_is_rejected	PASSOU
75	IoT · Handshake mTLS	test_broker_without_matching_san_is_rejected_by_device	PASSOU
76	IoT · Handshake mTLS	test_hostname_mismatch_is_rejected	PASSOU
77	IoT · Handshake mTLS	test_client_without_certificate_is_rejected	PASSOU